

```
!pip install youtube-transcript-api
```

Requirement already satisfied: youtube-transcript-api in /usr/local/lib/python3.12/dist-packages (1.2.4)
Requirement already satisfied: defusedxml<0.8.0,>=0.7.1 in /usr/local/lib/python3.12/dist-packages (from youtube-transcript-api) (0.7.1)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from youtube-transcript-api) (2.32.4)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->youtube-transcript-api) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->youtube-transcript-api) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->youtube-transcript-api) (2.2.3)
Requirement already satisfied: certifi<2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->youtube-transcript-api) (2024.7.4)

```
from youtube_transcript_api import YouTubeTranscriptApi  
  
print("Library Installed Successfully")
```

```
Library Installed Successfully
```

```
from youtube_transcript_api import YouTubeTranscriptApi  
  
video_id = "Rni7Fz7208c"  
  
ytt_api = YouTubeTranscriptApi()  
  
transcript = ytt_api.fetch(video_id)  
  
print("Total transcript entries:", len(transcript))  
  
print("\nFirst 5 transcript entries:\n")  
  
for item in transcript[:5]:  
    print(item)
```

```
Total transcript entries: 2839
```

```
First 5 transcript entries:
```

```
FetchTranscriptSnippet(text='Heat. Heat.', start=0.0, duration=3.0)  
FetchTranscriptSnippet(text='[music]', start=11.79, duration=2.02)  
FetchTranscriptSnippet(text='[music]', start=21.615, duration=4.504)  
FetchTranscriptSnippet(text='Our', start=23.119, duration=3.0)  
FetchTranscriptSnippet(text='[music]', start=27.51, duration=2.02)
```

```
from youtube_transcript_api import YouTubeTranscriptApi  
  
print(dir(YouTubeTranscriptApi))
```

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__getstate__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__']
```

```
from youtube_transcript_api import YouTubeTranscriptApi  
  
api = YouTubeTranscriptApi()  
  
transcript = api.fetch("Rni7Fz7208c")  
  
print(type(transcript))  
print("Transcript fetched successfully!")  
  
print("\nFirst entry:\n")  
print(transcript[0])
```

```
<class 'youtube_transcript_api.transcripts.FetchedTranscript'>  
Transcript fetched successfully!
```

```
First entry:
```

```
FetchTranscriptSnippet(text='Heat. Heat.', start=0.0, duration=3.0)
```

```
from youtube_transcript_api import YouTubeTranscriptApi  
  
api = YouTubeTranscriptApi()  
  
transcript = api.fetch("Rni7Fz7208c")  
  
with open("transcript.txt", "w", encoding="utf-8") as f:  
    for item in transcript:  
        f.write(f"[{item.start:.2f}] {item.text}\n")  
  
print("Transcript saved to transcript.txt")  
print("Total transcript entries:", len(transcript))
```

Transcript saved to transcript.txt
Total transcript entries: 2839

```
chunks = []
chunk_size = 20

for i in range(0, len(transcript), chunk_size):
    chunk = transcript[i:i + chunk_size]

    start_time = chunk[0].start

    text = " ".join([entry.text for entry in chunk])

    chunks.append({
        "timestamp": start_time,
        "text": text
    })

print("Total Chunks Created:", len(chunks))

print("\nFirst Chunk:\n")
print(chunks[0])
```

Total Chunks Created: 142

First Chunk:

```
{'timestamp': 0.0, 'text': 'Heat. Heat. [music] [music] Our [music] [music] [music] audience is largely wannabe entrepreneur
```

```
!pip install sentence-transformers faiss-cpu
```

```
Requirement already satisfied: sentence-transformers in /usr/local/lib/python3.12/dist-packages (5.5.1)
Collecting faiss-cpu
  Using cached faiss_cpu-1.14.2-cp310-abi3-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (7.8 kB)
Requirement already satisfied: transformers<6.0.0,>=4.41.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.41.0)
Requirement already satisfied: huggingface-hub>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (0.23.0)
Requirement already satisfied: torch>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (2.11.0)
Requirement already satisfied: numpy>=1.20.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (2.0.2)
Requirement already satisfied: scikit-learn>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (1.3.2)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (1.16.0)
Requirement already satisfied: typing_extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.6.3)
Requirement already satisfied: tqdm>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.67.3)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from faiss-cpu) (26.2)
Requirement already satisfied: click>=8.4.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers) (8.1.8)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers) (3.10.0)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers) (2023.12.1)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers) (1.4.3)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers) (0.23.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers) (6.0.2)
Requirement already satisfied: typer<0.26.0,>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers) (0.26.0)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.22.0->sentence-transformers) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.22.0->sentence-transformers) (3.5.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (75.0.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (1.13.3)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (3.3)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (3.1.3)
Requirement already satisfied: regex>=2025.10.22 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0) (2025.10.22)
Requirement already satisfied: tokenizers<0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0) (0.22.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0) (0.5.1)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.23.0) (4.7.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.23.0) (2025.1.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.23.0) (1.0.7)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.23.0) (3.10)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub>=0.23.0) (0.14.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.11.0->sentence-transformers) (1.3.0)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0->huggingface-hub>=0.23.0) (1.5.1)
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0->huggingface-hub>=0.23.0) (13.8.0)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (0.0.2)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch>=1.11.0->sentence-transformers) (3.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer<0.26.0,>=0.20.0->huggingface-hub>=0.23.0) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer<0.26.0,>=0.20.0->huggingface-hub>=0.23.0) (2.18.0)
Requirement already satisfied: mdurl~>0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=13.8.0->typer<0.26.0,>=0.20.0->huggingface-hub>=0.23.0) (0.1.2)
Using cached faiss_cpu-1.14.2-cp310-abi3-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (18.2 MB)
Installing collected packages: faiss-cpu
Successfully installed faiss-cpu-1.14.2
```

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

print("Embedding model loaded successfully!")
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
modules.json: 100% 349/349 [00:00<00:00, 28.4kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 10.5kB/s]

README.md: 100% 10.5k/10.5k [00:00<00:00, 645kB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 4.62kB/s]

config.json: 100% 612/612 [00:00<00:00, 52.0kB/s]

model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 76.2MB/s]

Loading weights: 100% 103/103 [00:00<00:00, 3341.25it/s]

tokenizer_config.json: 100% 350/350 [00:00<00:00, 31.1kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 6.69MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 15.4MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 6.57kB/s]

config.json: 100% 190/190 [00:00<00:00, 13.9kB/s]

Embedding model loaded successfully!

```

```

chunk_texts = [chunk["text"] for chunk in chunks]

embeddings = model.encode(
    chunk_texts,
    show_progress_bar=True
)

print("Total Embeddings:", len(embeddings))
print("Embedding Dimension:", len(embeddings[0]))

```

```

Batches: 100% 5/5 [00:20<00:00, 3.16s/it]

Total Embeddings: 142
Embedding Dimension: 384

```

```

print(type(embeddings))
print(embeddings.shape)

```

```

<class 'numpy.ndarray'>
(142, 384)

```

```

import faiss
import numpy as np

embeddings = np.array(embeddings).astype("float32")

dimension = embeddings.shape[1]

index = faiss.IndexFlatL2(dimension)

index.add(embeddings)

print("FAISS Index Created Successfully!")
print("Total Vectors Stored:", index.ntotal)

```

```

FAISS Index Created Successfully!
Total Vectors Stored: 142

```

```

query = "What is first principles thinking?"

query_embedding = model.encode([query]).astype("float32")

distances, indices = index.search(query_embedding, k=3)

print("Top Matching Chunks:\n")

for idx in indices[0]:
    print("=" * 80)
    print("Timestamp:", chunks[idx]["timestamp"])
    print(chunks[idx]["text"][:500])

```

Top Matching Chunks:

```

=====
Timestamp: 3622.96
uh, you know, like what what do you want to major in type of thing. Uh [laughter] it's usually like they you get you get giv
=====
Timestamp: 6704.56
So, that's that's the the main thing you should aim for. Aim to make more than you take. Um be a be a you know a net contrit
=====
Timestamp: 4562.96
curiosity. I I mean I think those are the three most important things for AI. >> Can you explain? Well, the truth said truth

```

```

def ask_podcast(question, top_k=3):

    query_embedding = model.encode([question]).astype("float32")

    distances, indices = index.search(query_embedding, k=top_k)

    best_idx = indices[0][0]

    result = chunks[best_idx]

    print("="*80)
    print("QUESTION:")
    print(question)

    print("\nBEST TIMESTAMP:")
    print(result["timestamp"])

    print("\nRELEVANT TRANSCRIPT:")
    print(result["text"][:1500])

    return result

```

ask_podcast("What advice does Elon Musk give to entrepreneurs?")

```

=====
QUESTION:
What advice does Elon Musk give to entrepreneurs?

BEST TIMESTAMP:
1198.96

RELEVANT TRANSCRIPT:
daily fluctuations, >> right? What's got you most excited now, Elon, in terms of all that you're building? You're doing so m
{'timestamp': 1198.96,
 'text': "daily fluctuations, >> right? What's got you most excited now, Elon, in terms of all that you're building? You're
doing so much. So, let me just preface and contextualize who is watching this. Uh, our audience is largely wannabe
entrepreneurs in India. >> Okay. uh really ambitious, really hungry, want to take the risk and build something and I feel
like all of us have so much to learn from you because you've done it so many times over in so many different domains. >>
Yeah. >> Uh so we will speak to them today and I will try and center all my questions in"}

```

```

def seconds_to_timestamp(seconds):
    hours = int(seconds // 3600)
    minutes = int((seconds % 3600) // 60)
    secs = int(seconds % 60)

    return f"{hours:02d}:{minutes:02d}:{secs:02d}"

```

print(seconds_to_timestamp(3622.96))

01:00:22

ask_podcast("What advice does Elon Musk give to entrepreneurs?")

```

=====
QUESTION:
What advice does Elon Musk give to entrepreneurs?

```

BEST TIMESTAMP:
1198.96

RELEVANT TRANSCRIPT:

```
daily fluctuations, >> right? What's got you most excited now, Elon, in terms of all that you're building? You're doing so much. So, let me just preface and contextualize who is watching this. Uh, our audience is largely wannabe entrepreneurs in India. >> Okay. uh really ambitious, really hungry, want to take the risk and build something and I feel like all of us have so much to learn from you because you've done it so many times over in so many different domains. >> Yeah. >> Uh so we will speak to them today and I will try and center all my questions in"}

```

```
!pip install transformers torch accelerate sentencepiece

```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.9.0)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.11.0+cpu)
Requirement already satisfied: accelerate in /usr/local/lib/python3.12/dist-packages (1.13.0)
Requirement already satisfied: sentencepiece in /usr/local/lib/python3.12/dist-packages (0.2.1)
Requirement already satisfied: huggingface-hub<2.0,>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1.5.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex>=2025.10.22 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.0)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from transformers) (0.25.1)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate) (5.9.5)
Requirement already satisfied: click>=8.4.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.5.0->transformers) (8.1.8)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.5.0->transformers) (1.2.2)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.5.0->transformers) (0.27.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.37.0)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (0.0.2)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers) (4.7.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers) (2025.11.12)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers) (1.0.7)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers) (3.10.1)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers) (0.14.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer->transformers) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer->transformers) (2.19.1)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=13.8.0->typer->transformers) (0.1.2)

```

```
pipeline("text-generation")

```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_3568/1081193035.py in <cell line: 0>()
----> 1 pipeline("text-generation")

NameError: name 'pipeline' is not defined

```

```
def ask_podcast(question, top_k=3):

    query_embedding = model.encode([question]).astype("float32")

    distances, indices = index.search(query_embedding, k=top_k)

    best_idx = indices[0][0]

    result = chunks[best_idx]

    timestamp = result["timestamp"]

    hours = int(timestamp // 3600)
    minutes = int((timestamp % 3600) // 60)
    seconds = int(timestamp % 60)

    readable_time = f"{hours:02d}:{minutes:02d}:{seconds:02d}"

    youtube_link = f"https://www.youtube.com/watch?v=Rni7Fz7208c&t={int(timestamp)}s"

    print(f"={80}")

```

```

print("QUESTION:")
print(question)

print("\nTIMESTAMP:")
print(readable_time)

print("\nVIDEO LINK:")
print(youtube_link)

print("\nANSWER:")
print(result["text"][:1000])

return result

```

```
ask_podcast("What advice does Elon Musk give to entrepreneurs?")
```

```

def ask_podcast(question, top_k=5):

    query_embedding = model.encode([question]).astype("float32")

    distances, indices = index.search(query_embedding, k=top_k)

    print("="*80)
    print("QUESTION:", question)

    print("\nTOP MATCHES\n")

    for rank, idx in enumerate(indices[0], start=1):

        timestamp = chunks[idx]["timestamp"]

        hours = int(timestamp // 3600)
        minutes = int((timestamp % 3600) // 60)
        seconds = int(timestamp % 60)

        readable_time = f"{hours:02d}:{minutes:02d}:{seconds:02d}"

        print("\n" + "="*80)
        print(f"Match #{rank}")
        print("Timestamp:", readable_time)
        print("Video Link:",
              f"https://www.youtube.com/watch?v=Rni7Fz7208c&t={int(timestamp)}s")

        print("\nTranscript:")
        print(chunks[idx]["text"][:500])

```

```
ask_podcast("What advice does Elon Musk give to entrepreneurs?")
```

```
!pip install gradio
```

```

import gradio as gr

def podcast_bot(question):

    query_embedding = model.encode([question]).astype("float32")

    distances, indices = index.search(query_embedding, k=5)

    best_idx = indices[0][0]

    result = chunks[best_idx]

    timestamp = result["timestamp"]

    hours = int(timestamp // 3600)
    minutes = int((timestamp % 3600) // 60)
    seconds = int(timestamp % 60)

    readable_time = f"{hours:02d}:{minutes:02d}:{seconds:02d}"

    youtube_link = f"https://www.youtube.com/watch?v=Rni7Fz7208c&t={int(timestamp)}s"

    return f"""
Answer:

{result['text'][:800]}

Timestamp:
{readable_time}

```

```

Video Link:
{youtube_link}
"""

demo = gr.Interface(
    fn=podcast_bot,
    inputs="text",
    outputs="text",
    title="Elon Musk Podcast Q&A Bot",
    description="Ask questions about the podcast and receive answers with timestamps."
)

demo.launch()

```

```
best_idx = indices[0][0]
```

```

def podcast_bot(question):

    query_embedding = model.encode([question]).astype("float32")

    distances, indices = index.search(query_embedding, k=5)

    best_idx = indices[0][0]

    result = chunks[best_idx]

    timestamp = result["timestamp"]

    hours = int(timestamp // 3600)
    minutes = int((timestamp % 3600) // 60)
    seconds = int(timestamp % 60)

    readable_time = f"{hours:02d}:{minutes:02d}:{seconds:02d}"

    youtube_link = f"https://www.youtube.com/watch?v=Rni7Fz7208c&t={int(timestamp)}s"

    answer = result["text"][:700]

    return f"""
Answer:
{answer}

Timestamp:
{readable_time}

Video Link:
{youtube_link}
"""

```

```
demo.launch()
```

```
best_idx = indices[0][0]
```

```
demo.launch()
```

```

import gradio as gr

demo = gr.Interface(
    fn=podcast_bot,
    inputs=gr.Textbox(
        label="Ask a Question",
        lines=2
    ),
    outputs=gr.Textbox(
        label="Answer",
        lines=15
    ),
    title="Elon Musk Podcast Q&A Bot",
    description="Ask questions about the podcast and receive answers with timestamps."
)

demo.launch()

```

```

def podcast_bot(question):

    query_embedding = model.encode([question]).astype("float32")

```

```

distances, indices = index.search(query_embedding, k=5)

best_idx = indices[0][0]
result = chunks[best_idx]

timestamp = result["timestamp"]

hours = int(timestamp // 3600)
minutes = int((timestamp % 3600) // 60)
seconds = int(timestamp % 60)

readable_time = f"{hours:02d}:{minutes:02d}:{seconds:02d}"

youtube_link = f"https://www.youtube.com/watch?v=Rni7Fz7208c&t={int(timestamp)}s"

answer = result["text"][:1000]

return answer, readable_time, youtube_link

demo = gr.Interface(
    fn=podcast_bot,
    inputs=gr.Textbox(label="Question"),
    outputs=[
        gr.Textbox(label="Answer", lines=12),
        gr.Textbox(label="Timestamp"),
        gr.Textbox(label="YouTube Link")
    ],
    title="Elon Musk Podcast Q&A Bot"
)

demo.launch()

```

```
demo.launch()
```

```

import os

print(os.path.exists("transcript.txt"))

```

```

from google.colab import files
files.download("transcript.txt")

```

```

import gradio as gr

demo = gr.Interface(
    fn=podcast_bot,
    inputs=gr.Textbox(label="Question"),
    outputs=[
        gr.Textbox(label="Answer", lines=12),
        gr.Textbox(label="Timestamp"),
        gr.Textbox(label="YouTube Link")
    ],
    title="Elon Musk Podcast Q&A Bot"
)

demo.launch()

```

```
print(type(index))
```

```
<class 'faiss.swigfaiss.IndexFlatL2'>
```

```
print(chunks[0])
```

```
{'timestamp': 0.0, 'text': "Heat. Heat. [music] [music] Our [music] [music] [music] audience is largely wannabe entrepreneur
```

```
print(embeddings.shape)
```

```
(142, 384)
```

```

from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")

def answer_question(question):
    query_embedding = model.encode([question])

```



```

D, I = index.search(
    np.array(query_embedding).astype("float32"),
    k=1
)

best_chunk = chunks[I[0][0]]

timestamp = int(best_chunk["timestamp"])

hours = timestamp // 3600
minutes = (timestamp % 3600) // 60
seconds = timestamp % 60

formatted_time = f"{hours:02}:{minutes:02}:{seconds:02}"

youtube_link = (
    f"https://www.youtube.com/watch?v=Rni7Fz7208c&t={timestamp}s"
)

return f"""
QUESTION:
{question}

TIMESTAMP:
{formatted_time}

VIDEO LINK:
{youtube_link}

ANSWER:
{best_chunk['text']}
"""

```

Loading weights: 100%

103/103 [00:00<00:00, 3226.00it/s]

```

import gradio as gr

demo = gr.Interface(
    fn=answer_question,
    inputs=gr.Textbox(label="Ask a Question"),
    outputs=gr.Textbox(label="Answer", lines=15),
    title="Podcast Q&A Bot"
)

```

```
demo.launch()
```

It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share`

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()

* Running on public URL: <https://4cdaea3724c7565557.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the



No interface is running right now

